



南开大学  
Nankai University

南 开 大 学

数据安全实验报告

---

## 实验 4：频率隐藏 OPE 方案实现

---

学院：密码与网络空间安全学院

专业：物联网工程

学号：2310422

姓名：谢小珂

指导教师：刘哲理

2026 年 6 月 6 日

# 目录

|                                |           |
|--------------------------------|-----------|
| <b>1 实验要求与完成情况</b>             | <b>1</b>  |
| <b>2 实验原理</b>                  | <b>1</b>  |
| 2.1 普通 OPE 的频率泄露问题 . . . . .   | 1         |
| 2.2 系统架构 . . . . .             | 1         |
| 2.3 插入与查询流程 . . . . .          | 2         |
| <b>3 实验环境与工程配置</b>             | <b>2</b>  |
| 3.1 工程文件与依赖 . . . . .          | 2         |
| 3.2 MySQL 服务与 UDF 加载 . . . . . | 2         |
| <b>4 核心代码实现</b>                | <b>4</b>  |
| 4.1 随机化加密 . . . . .            | 4         |
| 4.2 本地频率表与随机插入位置 . . . . .     | 4         |
| 4.3 服务端编码生成与编码更新 . . . . .     | 5         |
| 4.4 SQL 存储过程中的同步更新 . . . . .   | 5         |
| <b>5 实验过程与结果分析</b>             | <b>6</b>  |
| 5.1 相同数字 5 连续插入 20 次 . . . . . | 6         |
| 5.2 CSV 结果核对 . . . . .         | 7         |
| 5.3 批量 workload 对照实验 . . . . . | 8         |
| 5.4 图表分析 . . . . .             | 8         |
| 5.5 textbook 参数对照 . . . . .    | 10        |
| <b>6 实验问题与解决方法</b>             | <b>11</b> |
| 6.1 UDF 状态需要重置 . . . . .       | 11        |
| 6.2 小规模实验不一定触发编码更新 . . . . .   | 11        |
| 6.3 查询正确性不能只看返回顺序 . . . . .    | 11        |
| <b>7 实验结论</b>                  | <b>12</b> |

## 1 实验要求与完成情况

本实验参照教材 6.3.3 中频率隐藏顺序保持加密的实现思路，完成客户端、MySQL UDF 与服务端编码树的协同实现，并在 `client.py` 中构造连续插入相同数值的测试用例。实验重点包括：相同明文多次插入时是否生成不同密文、是否被分散到不同顺序编码位置、编码树是否发生分裂或编码更新，以及更新后范围查询是否仍保持正确。

本次实现的工程由 Python 客户端、C++ 编码树、MySQL UDF、SQL 存储过程和结果分析脚本组成。实验中首先使用观察参数 `FHOPE_M=4` 与 `FHOPE_INITIAL_UPPER=1<<8` 放大小规模数据下的结构变化；随后使用接近教材的大参数进行对照，验证在大编码空间下小规模数据通常不触发分裂但查询仍正确。

从实验结果看，主测试 `same-number` 连续插入 20 次数字 5，得到 `unique_plaintexts=1`、`unique_ciphertexts=20`、`unique_inserted_encodings=20`，说明相同明文没有表现为重复密文或单一固定编码；查询区间 `[5,5]` 返回 20 条记录且 `correct=YES`。批量 workload 中，`increasing_20` 与 `random_small_domain_50` 进一步触发了编码更新，说明编码更新逻辑也被实际覆盖。

## 2 实验原理

### 2.1 普通 OPE 的频率泄露问题

顺序保持加密 (Order-Preserving Encryption, OPE) 要求密文或编码能够保留明文大小关系，因此数据库可以在不了解明文内容的情况下执行排序和范围查询。然而，若相同明文总是对应相同密文或相同编码，服务器可直接根据重复密文频数推断明文频率分布。例如某一密文重复出现 20 次时，服务器虽不能解密，但可以确定某个明文值也重复出现了 20 次，这与频率隐藏目标相冲突。

FH-OPE 的目标是在保留范围查询能力的同时，削弱相同明文带来的直接频率泄露。本实验中采用两层随机化：第一，客户端每次对明文执行随机化 AES 加密，相同明文也会生成不同密文；第二，客户端维护本地频率表 `local_table`，在相同明文对应的合法逻辑区间内随机选择插入位置，使相同明文分散到不同的顺序编码位置。

### 2.2 系统架构

本实验系统结构如图 1 所示。客户端负责明文侧操作，包括明文频率统计、随机插入位置计算、随机加密和范围查询边界计算；MySQL 仅保存 `encoding` 与 `ciphertext`；C++ UDF 在数据库侧维护编码树，负责将逻辑位置映射为可排序编码，并在容量不足或编码空间不足时触发节点分裂或编码更新。

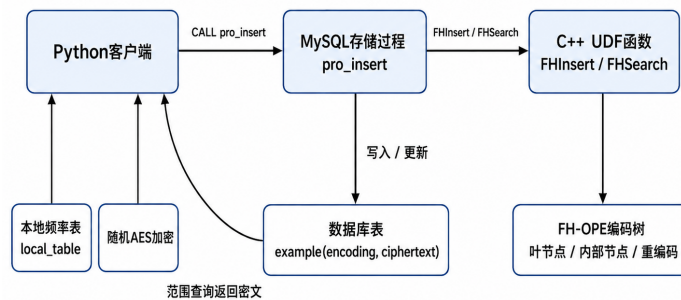


图 1: FH-OPE 实验系统架构

## 2.3 插入与查询流程

插入时, 客户端先计算小于当前明文的记录数 `presum`。若当前明文首次出现, 则插入位置为 `presum`; 若已出现多次, 则先将出现次数加一, 再在 `[presum, presum + count - 1]` 内随机选择逻辑位置。随后客户端随机加密明文, 并通过 `CALL pro_insert(pos, ciphertext)` 提交给数据库。服务端编码树根据逻辑位置定位叶节点, 生成新的顺序编码并插入表中。

查询时, 客户端由 `local_table` 计算查询左端点和右端点对应的逻辑位置边界, 调用 `FHSearch` 得到编码区间, 再由 MySQL 执行 `encoding` 范围查询。客户端解密返回密文, 并使用多重集合比较期望结果与实际结果, 避免因返回顺序不同而误判。

# 3 实验环境与工程配置

## 3.1 工程文件与依赖

实验在 Ubuntu 虚拟机环境中完成, 工程包含 `client.py`、`load.sql`、`src/Node.cpp`、`src/UDF.cpp`、`scripts/build_udf.sh`、`scripts/run_experiments.sh` 与结果目录 `results`。Python 依赖包括 PyMySQL、pycryptodome、pandas、matplotlib 等。图 2 显示了工程目录和依赖安装结果。

```
xxk@xxk-VMware-Virtual-Platform: ~/Desktop/fhope_better_project

# 查看工程文件
ls -la

# 确认 pip 依赖已安装
pip list
/home/xxk/Desktop/fhope_better_project
total 56
drwxrwxr-x 6 xxk xxk 4096 Jun 6 13:10 .
drwxr-xr-x 7 xxk xxk 4096 Jun 6 13:09 ..
-rwxr-xr-x 1 xxk xxk 14046 Jun 6 03:56 client.py
-rw-r--r-- 1 xxk xxk 46 Jun 6 03:56 .gitignore
-rw-r--r-- 1 xxk xxk 1905 Jun 6 03:56 load.sql
-rw-r--r-- 1 xxk xxk 1758 Jun 6 03:56 README.md
-rw-r--r-- 1 xxk xxk 39 Jun 6 03:56 requirements.txt
drwxr-xr-x 2 xxk xxk 4096 Jun 6 03:56 results
drwxr-xr-x 2 xxk xxk 4096 Jun 6 03:56 scripts
drwxr-xr-x 2 xxk xxk 4096 Jun 6 03:56 src
drwxrwxr-x 6 xxk xxk 4096 Jun 6 13:22 .venv

Package            Version
-----
contourpy           1.3.3
cycler              0.12.1
fonttools           4.63.0
kiwisolver           1.5.0
matplotlib           3.10.9
numpy               2.4.6
packaging            26.2
pandas              3.0.3
pillow              12.2.0
pip                 24.0
pycryptodome         3.23.0
PyMySQL             1.2.0
pyparsing            3.3.2
python-dateutil      2.9.0.post0
six                 1.17.0
(.venv) xxk@xxk-VMware-Virtual-Platform: ~/Desktop/fhope_better_project$
```

图 2: 工程文件结构与 Python 依赖

## 3.2 MySQL 服务与 UDF 加载

实验使用 MySQL 8.0.46。首先启动 MySQL 服务, 创建 `fhope` 用户并授予权限, 然后测试数据库连接。图 3 中可以看到用户创建、授权与版本查询均成功。

```

xk@xk-VMware-Virtual-Platform: ~/Desktop/fhope_better_project
six 1.17.0
(.venv) xk@xk-VMware-Virtual-Platform:~/Desktop/fhope_better_project$ sudo service mysql start
(.venv) xk@xk-VMware-Virtual-Platform:~/Desktop/fhope_better_project$ sudo mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.46-0ubuntu0.24.04.2 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE USER IF NOT EXISTS 'fhope'@'localhost' IDENTIFIED BY '123456';
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE USER IF NOT EXISTS 'fhope'@'127.0.0.1' IDENTIFIED BY '123456';
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> GRANT ALL PRIVILEGES ON *.* TO 'fhope'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT ALL PRIVILEGES ON *.* TO 'fhope'@'127.0.0.1';
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)

mysql> EXIT;
Bye
(.venv) xk@xk-VMware-Virtual-Platform:~/Desktop/fhope_better_project$ mysql -ufhope -p123456 -e "SELECT VERSION();"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| VERSION() |
+-----+
| 8.0.46-0ubuntu0.24.04.2 |
+-----+
(.venv) xk@xk-VMware-Virtual-Platform:~/Desktop/fhope_better_project$

```

图 3: MySQL 服务启动、用户授权与版本验证

随后运行 `scripts/build_udf.sh observation` 编译 `libfhope.so` 并复制到 MySQL 插件目录。图 4 显示动态库已安装到 `/usr/lib/mysql/plugin/libfhope.so`, 且 MySQL 的 `plugin_dir` 与安装路径一致。

```

[build] installed /usr/lib/mysql/plugin/libfhope.so
(.venv) xk@xk-VMware-Virtual-Platform:~/Desktop/fhope_better_project$ sudo find /usr/lib -name "*.so" 2>/dev/null
grep -E "(fh|hope)" | head -5
/usr/lib/mysql/plugin/libfhope.so
/usr/lib/x86_64-linux-gnu/libmpi_mpifh-gfortran.so
/usr/lib/x86_64-linux-gnu/libmpi_mpifh.so
/usr/lib/x86_64-linux-gnu/openmpi/lib/libmpi_mpifh.so
/usr/lib/x86_64-linux-gnu/fortran/gfortran/libmpi_mpifh.so
(.venv) xk@xk-VMware-Virtual-Platform:~/Desktop/fhope_better_project$ mysql -ufhope -p123456 -e "SHOW VARIABLES LIKE 'plugin_dir';"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| Variable_name | Value |
+-----+
| plugin_dir    | /usr/lib/mysql/plugin/ |
+-----+

```

图 4: UDF 动态库编译安装与插件目录确认

执行 `mysql -ufhope -p123456 < load.sql` 后, 数据库、表、UDF 函数和存储过程被创建。图 5 中 `FHReset()` 返回 1, 叶节点分裂、内部节点分裂和总记录数均为 0, 说明编码树初始状态正确。

```
(.venv) xxk@xxk-VMware-Virtual-Platform:~/Desktop/fhope_better_project$ mysql -ufhope -p123456 < load.sql
mysql: [Warning] Using a password on the command line interface can be insecure.
(.venv) xxk@xxk-VMware-Virtual-Platform:~/Desktop/fhope_better_project$ mysql -ufhope -p123456 -D fhope_lab -e "SELECT FHReset(), FHLeafSplits(), FHInternalSplits(), FHTotal();"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+-----+-----+-----+
| FHReset() | FHLeafSplits() | FHInternalSplits() | FHTotal() |
+-----+-----+-----+-----+
| 1 | 0 | 0 | 0 |
+-----+-----+-----+-----+
```

图 5: SQL 加载与 UDF 初始状态测试

## 4 核心代码实现

### 4.1 随机化加密

相同明文能否生成不同密文，是频率隐藏的第一层保障。客户端每次加密时生成新的 16 字节随机 IV，再将 iv + inner 包装加密为最终密文。数据库只能看到 Base64 编码后的随机密文，无法通过重复密文直接统计明文频率。

Listing 1: 随机化 AES 加密与解密

```
1 def random_encrypt(plaintext: Any) -> str:
2     raw = plaintext_text(plaintext).encode("utf-8")
3     iv = get_random_bytes(16)
4     inner = aes_enc(raw, iv)
5     outer = aes_enc(iv + inner, BASE_IV)
6     return base64.b64encode(outer).decode("utf-8")
7
8 def random_decrypt(ciphertext: str) -> str:
9     outer = base64.b64decode(ciphertext.encode("utf-8"))
10    plaintext = aes_dec(outer, BASE_IV)
11    inner_iv = plaintext[:16]
12    inner_ct = plaintext[16:]
13    return aes_dec(inner_ct, inner_iv).decode("utf-8")
```

### 4.2 本地频率表与随机插入位置

相同明文的随机插入位置由 cal\_pos 决定。连续插入数字 5 时，小于 5 的记录数为 0，但 local\_table[5] 从 1 增长到 20，因此每次会在不断扩大的合法区间中随机选择位置。该设计使相同明文不会简单追加到同一固定位置。

Listing 2: 相同明文的随机插入位置计算

```
1 def cal_pos(plaintext: Any) -> tuple[int, dict[Any, int], dict[Any, int]]:
2     before = sorted_table_snapshot()
3     presum = sum(v for k, v in local_table.items() if k < plaintext)
4
5     if plaintext in local_table:
6         local_table[plaintext] += 1
7         count = local_table[plaintext]
8         pos = random.randint(presum, presum + count - 1)
9     else:
10        local_table[plaintext] = 1
```

```

11     pos = presum
12
13     after = sorted_table_snapshot()
14     return pos, before, after

```

### 4.3 服务端编码生成与编码更新

服务端叶节点根据插入位置左右相邻编码的空隙生成新编码。若空隙不足, 即  $right-left < 2$ , 则触发 recode, 重新分配一定区间内记录的编码, 并通过 FHUpdate 同步到数据库表。

Listing 3: 叶节点编码生成与重编码触发

```

1 long long LeafNode::encode(int pos) {
2     long long left = lower;
3     long long right = upper;
4
5     if (pos > 0) {
6         left = encoding[pos - 1];
7     }
8     if (pos < static_cast<int>(encoding.size()) - 1) {
9         right = encoding[pos + 1];
10    }
11
12    if ((right - left) < 2) {
13        std::vector<LeafNode *> node_list;
14        node_list.push_back(this);
15        recode(node_list);
16        return 0;
17    }
18
19    long long frag = (right - left) / 2;
20    encoding[pos] = right - frag;
21    return encoding[pos];
22 }

```

### 4.4 SQL 存储过程中的同步更新

当 FHInsert 返回非 0 时, 新记录直接获得有效编码; 当返回 0 时, 说明编码树已发生重编码, 此时表中已有记录的编码可能过期, 必须调用 FHUpdate(ciphertext) 批量同步。该同步逻辑是范围查询正确性的关键。

Listing 4: pro\_insert 中的编码更新同步

```

1 CREATE PROCEDURE pro_insert(IN pos BIGINT, IN ct VARCHAR(1024))
2 BEGIN
3     DECLARE code BIGINT DEFAULT 0;
4
5     SET code = FHInsert(pos, ct);
6     INSERT INTO example(encoding, ciphertext) VALUES(code, ct);
7

```

```

8      IF code = 0 THEN
9          UPDATE example
10         SET encoding = FHUpdate(ciphertext)
11         WHERE (encoding >= FHStart() AND encoding < FHEnd())
12             OR encoding = 0;
13     END IF;
14 END

```

## 5 实验过程与结果分析

### 5.1 相同数字 5 连续插入 20 次

主实验运行命令为：

Listing 5: 相同数字多次插入实验命令

```
1 python3 client.py --mode same-number --count 20 --value 5 --run-name same_number_20
```

图 6 显示运行配置: mode=same-number, count=20, 所有输入值均为 5。程序开始前执行了本地表、MySQL 表与 FH-OPE 编码树重置, 确保本轮实验不受前一轮状态影响。

```

(.venv) xxk@xxk-VMware-Virtual-Platform:~/Desktop/fhope_better_project$ python3 client.py --mode same-number --count
20 --value 5 --run-name same_number_20
[config] run_name=same_number_20 mode=same-number count=20 seed=20260606
[config] values=[5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5]
[reset] local_table cleared, MySQL table truncated, FH-OPE tree reset
[same-number] insert #01 plaintext='5' pos=0 encoding=128 leaf=0->0 internal=0->0 update=NO changed_rows=0
[same-number] insert #02 plaintext='5' pos=0 encoding=64 leaf=0->0 internal=0->0 update=NO changed_rows=0
[same-number] insert #03 plaintext='5' pos=0 encoding=32 leaf=0->0 internal=0->0 update=NO changed_rows=0
[same-number] insert #04 plaintext='5' pos=3 encoding=192 leaf=0->1 internal=0->0 update=NO changed_rows=0
[same-number] insert #05 plaintext='5' pos=4 encoding=224 leaf=1->1 internal=0->0 update=NO changed_rows=0
[same-number] insert #06 plaintext='5' pos=1 encoding=48 leaf=1->1 internal=0->0 update=NO changed_rows=0

```

图 6: same-number 工作负载启动与前几次插入

图 7 展示了第 6 到第 16 次插入的日志。可以看到插入位置 pos 并非固定为 0, 而是在相同明文的合法区间内变化; 第 7、8、11、13、16 次附近出现叶节点分裂, 说明编码树结构随随机插入逐步调整。该实验最终得到 leaf\_splits=7、internal\_splits=2。

```

[same-number] insert #06 plaintext='5' pos=1 encoding=48 leaf=1->1 internal=0->0 update=NO changed_rows=0
[same-number] insert #07 plaintext='5' pos=6 encoding=240 leaf=1->2 internal=0->0 update=NO changed_rows=0
[same-number] insert #08 plaintext='5' pos=2 encoding=56 leaf=2->3 internal=0->1 update=NO changed_rows=0
[same-number] insert #09 plaintext='5' pos=6 encoding=208 leaf=3->3 internal=1->1 update=NO changed_rows=0
[same-number] insert #10 plaintext='5' pos=1 encoding=40 leaf=3->3 internal=1->1 update=NO changed_rows=0
[same-number] insert #11 plaintext='5' pos=1 encoding=36 leaf=3->4 internal=1->1 update=NO changed_rows=0
[same-number] insert #12 plaintext='5' pos=4 encoding=52 leaf=4->4 internal=1->1 update=NO changed_rows=0
[same-number] insert #13 plaintext='5' pos=10 encoding=216 leaf=4->5 internal=1->1 update=NO changed_rows=0
[same-number] insert #14 plaintext='5' pos=13 encoding=248 leaf=5->5 internal=1->1 update=NO changed_rows=0
[same-number] insert #15 plaintext='5' pos=11 encoding=220 leaf=5->5 internal=1->1 update=NO changed_rows=0
[same-number] insert #16 plaintext='5' pos=10 encoding=212 leaf=5->6 internal=1->2 update=NO changed_rows=0

```

图 7: 连续插入数字 5 时的随机位置与节点分裂

插入完成后, 客户端对 [5, 5] 执行范围查询。图 8 中 pos\_range=[0, 20), 编码区间为 [32, 9223372036854775807), 返回 20 条记录。所有记录解密后均为 5, 且 expected=20 actual=20 correct=YES, 说明分裂后范围查询仍然正确。



```
[query] plaintext_range=[5,5] pos_range=[0,20) code_range=[32,9223372036854775807) returned=20
id=3 encoding=32 cipher_prefix=fF8Uh/NVIULFDpKEtnMb+kH plaintext=5
id=11 encoding=36 cipher_prefix=dUs84qenSCxUp93n+6+jD69E plaintext=5
id=10 encoding=40 cipher_prefix=Mg6PS30UWPb+cAuCy8pWnU/d plaintext=5
id=6 encoding=48 cipher_prefix=1wMUUSrlo5XX0mon+J9uf+Mv plaintext=5
id=12 encoding=52 cipher_prefix=RDZdcAQDeZ3AjQW+IhWwxJb plaintext=5
id=8 encoding=56 cipher_prefix=tLabdNfwh6WY3f0qCzqqGwoP plaintext=5
id=20 encoding=60 cipher_prefix=ENEVYkjDDBKBdtLKx+c8Nhg plaintext=5
id=2 encoding=64 cipher_prefix=VKVjmp6irql8qjFe1s03LY+e plaintext=5
id=1 encoding=128 cipher_prefix=eANBBA1n3ium3S80jnmKLXb2 plaintext=5
id=4 encoding=192 cipher_prefix=cuq5+g/65Wh66BT9pxKv31j2 plaintext=5
id=19 encoding=200 cipher_prefix=tjZQawYMMFk9+Uipk30x2eeQ plaintext=5
id=9 encoding=208 cipher_prefix=eZzIt+oID1loApDqkYf/g5rK plaintext=5
id=16 encoding=212 cipher_prefix=cYo4yW4VuF41oZKaZviyveg5 plaintext=5
id=17 encoding=214 cipher_prefix=7TmWYMXRNKc8e6GR245S8r5X plaintext=5
id=13 encoding=216 cipher_prefix=xfSWhC73K/lBS8aXQ3YwhEOi plaintext=5
id=15 encoding=220 cipher_prefix=QNv2x/q+MBnJzWXY7Z1qnhxy plaintext=5
id=5 encoding=224 cipher_prefix=043k8MCfT1ULiePVGb/z51vy plaintext=5
id=7 encoding=240 cipher_prefix=VwFFJadQm9EuHJgXSuDzDkHW plaintext=5
id=18 encoding=244 cipher_prefix=bMXQidyKuIpAB0AkjqAlBHKM plaintext=5
id=14 encoding=248 cipher_prefix=zE9QnqZdbKkhPGkxIVZJ/Cao plaintext=5
[query-check] expected=20 actual=20 correct=YES
```

图 8: same-number 范围查询正确性验证

图 9 给出了主实验的总结：唯一明文数为 1，唯一密文数为 20，唯一插入编码数为 20。这是频率隐藏效果的直接证据：数据库侧看不到 20 个完全相同的密文，也看不到同一明文被固定到单个编码位置。

```
[summary] unique_plaintexts=1 unique_ciphertexts=20 unique_inserted_encodings=20 leaf_splits=7 internal_splits=2 updates=0
[files] saved to results/same_number_20
```

图 9: same-number 终端汇总结果

## 5.2 CSV 结果核对

为避免仅凭终端输出判断，本实验同时保存 workload\_summary.csv 与 query\_results.csv。图 10 显示 same\_number\_20 的统计为：count=20、unique\_plaintexts=1、unique\_ciphertexts=20、unique\_inserted\_encodings=20、leaf\_splits=7、internal\_splits=2、encoding\_updates=0、query\_correct=1。

```
(.venv) xk@xk-VMware-Virtual-Platform:~/Desktop/fhope_better_project$ cat results/same_number_20/workload_summary.csv
run_name,workload,count,unique_plaintexts,unique_ciphertexts,unique_inserted_encodings,leaf_splits,internal_splits,encoding_updates,changed_rows_total,query_correct
same_number_20,same-number,20,1,20,20,7,2,0,0,1
```

图 10: same-number 的 workload\_summary.csv 结果

图 11 进一步显示查询左边界与右边界分别为 5 和 5，逻辑位置区间为 [0,20)，期望、实际和返回数量均为 20，query\_correct=1。这证明查询正确性不是人工观察，而是由程序自动比较多重集合后得到。

```
(.venv) xk@xk-VMware-Virtual-Platform:~/Desktop/fhope_better_project$ cat results/same_number_20/query_results.csv
workload,query_left,query_right,left_pos,right_pos,left_code,right_code,returned_count,expected_count,actual_count,query_correct,expected_counter,actual_counter
same-number,5,5,0,20,32,9223372036854775807,20,20,20,1,{'5': 20},{'5': 20}
```

图 11: same-number 的 query\_results.csv 结果

需要说明的是，在本次随机种子和观察参数下，same-number 主测试触发了叶节点与内部节点分裂，但未触发编码更新。这不是实验失败，因为编码更新只在局部编码间隙耗尽时发生，与随

机插入位置、当前树形和编码空间共同相关。为验证编码更新机制，后续批量 workload 继续运行了递增序列和小值域随机序列。

### 5.3 批量 workload 对照实验

批量脚本运行了 original、same-number、same-string、increasing、skewed 和 random-small-domain 六类 workload。汇总结果如图 12 与表 1 所示。所有 workload 的 query\_correct 均为 1，说明插入、分裂、重编码和查询流程整体正确。

```
(.venv) xxk@xxk-VMware-Virtual-Platform: ~/Desktop/fhope_better_project$ cat results/_summary/all_workload_summary.csv
run_name,workload,count,unique_plaintexts,unique_ciphertexts,unique_inserted_encodings,leaf_splits,internal_splits,encoding_updates,changed_rows_total,query_correct,run_dir
increasing_20,increasing,20,20,20,9,9,5,4,22,1,increasing_20
original_observation,original,8,5,8,2,0,0,0,1,original_observation
random_small_domain_50,random-small-domain,50,10,50,44,20,13,8,35,1,random_small_domain_50
same_number_20,same-number,20,1,20,20,7,2,0,0,1,same_number_20
same_string_20,same-string,20,1,20,20,7,2,0,0,1,same_string_20
skewed_20,skewed,20,9,20,20,8,4,0,0,1,skewed_20
```

图 12: 批量 workload 汇总 CSV

表 1: 不同 workload 的实验结果汇总

| workload            | 数量 | 唯一明文 | 唯一密文 | 唯一编码 | 叶分裂 | 内部分裂 | 更新数 | 正确  |
|---------------------|----|------|------|------|-----|------|-----|-----|
| original            | 8  | 5    | 8    | 8    | 2   | 0    | 0   | YES |
| same-number         | 20 | 1    | 20   | 20   | 7   | 2    | 0   | YES |
| same-string         | 20 | 1    | 20   | 20   | 7   | 2    | 0   | YES |
| skewed              | 20 | 9    | 20   | 20   | 8   | 4    | 0   | YES |
| increasing          | 20 | 20   | 20   | 9    | 9   | 5    | 4   | YES |
| random-small-domain | 50 | 10   | 50   | 44   | 20  | 13   | 8   | YES |

从表 1 可见, increasing\_20 触发了 4 次编码更新并影响 22 行记录, random\_small\_domain\_50 触发了 8 次编码更新并影响 35 行记录。因此, 虽然相同数字主测试没有触发编码更新, 但本实验的整体 workload 已经实际覆盖了编码更新路径。

### 5.4 图表分析

图 13 对比了不同 workload 的唯一明文、唯一密文和唯一编码数量。对于 same-number 和 same-string, 唯一明文数均为 1, 但唯一密文数和唯一编码数均为 20, 说明同一明文被随机加密和随机插入机制分散, 频率隐藏目标得到体现。

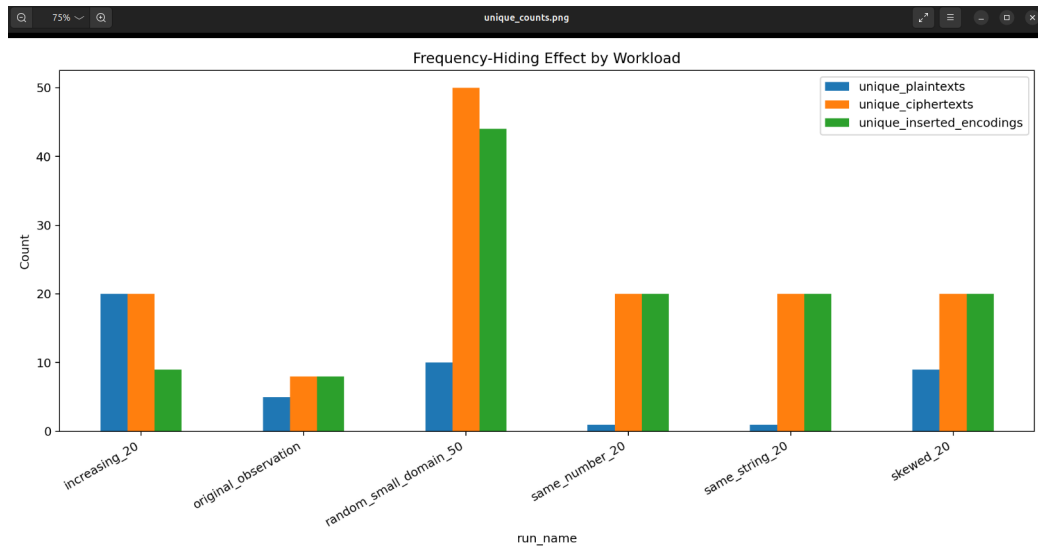


图 13: 不同 workload 的唯一明文、唯一密文和唯一编码数量

图 14 展示了各 workload 的叶节点与内部节点分裂次数。由于使用 observation 小参数，各类 20 条以上的 workload 都能观察到叶节点分裂；random-small-domain 插入 50 条记录，因此分裂次数最多。

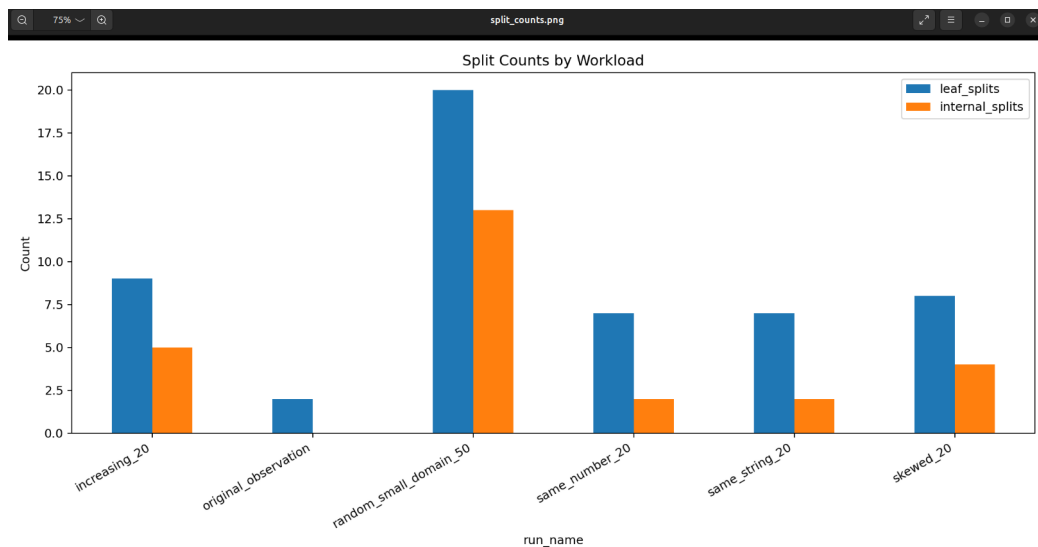


图 14: 不同 workload 的节点分裂次数

图 15 统计编码更新次数和被更新行数。结果表明，编码更新并非每个 workload 都会发生，而是在编码空间局部不足时触发；一旦触发，旧记录的编码会通过 FHUpdate 同步更新。

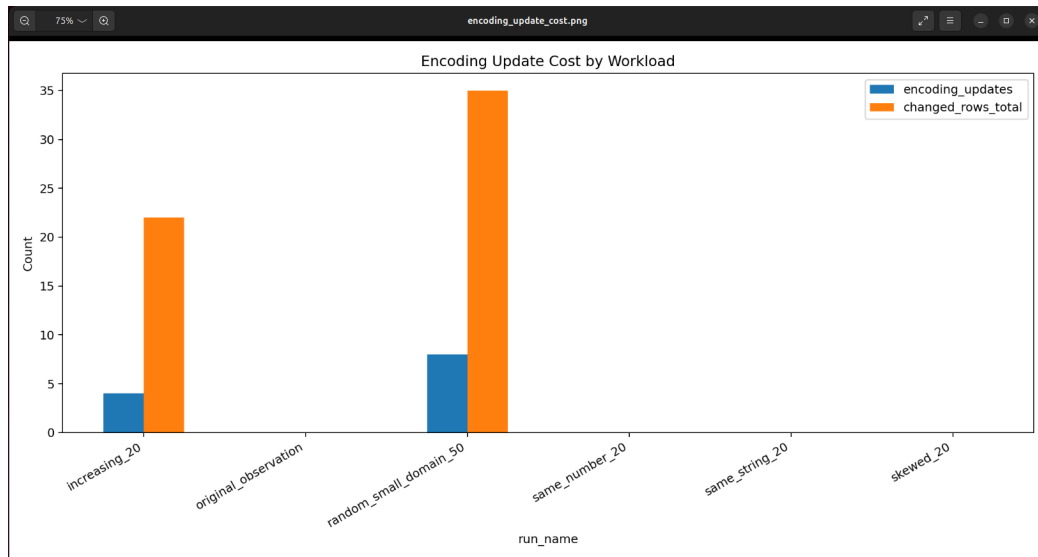


图 15: 不同 workload 的编码更新次数与更新代价

图 16 展示了插入序号与编码的关系。相同明文和偏斜分布并不是简单单调追加，而是在编码空间中呈现分散分布；这与随机插入位置、分裂和重编码共同作用有关。

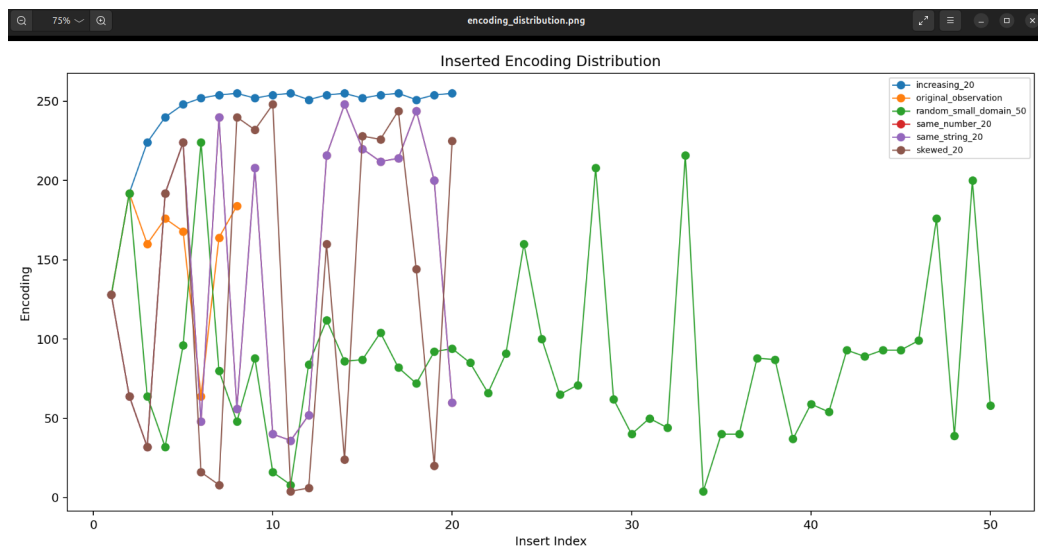


图 16: 插入序号与 encoding 分布

## 5.5 textbook 参数对照

为了说明参数选择对观察结果的影响,实验将 UDF 重新编译为 textbook 参数,即  $FHOPE\_M=128$  与  $FHOPE\_INITIAL\_UPPER=1 \ll 62$ 。图 17 显示了重新编译过程。

```
(.venv) xxk@xxk-VMware-Virtual-Platform:~/Desktop/fhope_better_project$ bash scripts/build_udf.sh textbook
[build] mode=textbook FHOPE_M=128 FHOPE_INITIAL_UPPER=(1LL<<62)
[build] plugin_dir=/usr/lib/mysql/plugin
In file included from src/UDF.cpp:1:
src/Node.h: In member function 'virtual long long int Node::insert(int, const std::string&)':
src/Node.h:27:34: warning: unused parameter 'pos' [-Wunused-parameter]
   27 |     virtual long long insert(int pos, const std::string &cipher) { return 0; }
      |                               ^~~~
src/Node.h:27:58: warning: unused parameter 'cipher' [-Wunused-parameter]
   27 |     virtual long long insert(int pos, const std::string &cipher) { return 0; }
      |                                     ^~~~~~
src/Node.h: In member function 'virtual long long int Node::search(int)':
src/Node.h:28:34: warning: unused parameter 'pos' [-Wunused-parameter]
   28 |     virtual long long search(int pos) { return 0; }
      |                               ^~~~
In file included from src/Node.cpp:1:
src/Node.h: In member function 'virtual long long int Node::insert(int, const std::string&)':
src/Node.h:27:34: warning: unused parameter 'pos' [-Wunused-parameter]
   27 |     virtual long long insert(int pos, const std::string &cipher) { return 0; }
      |                               ^~~~
src/Node.h:27:58: warning: unused parameter 'cipher' [-Wunused-parameter]
   27 |     virtual long long insert(int pos, const std::string &cipher) { return 0; }
      |                                     ^~~~~~
src/Node.h: In member function 'virtual long long int Node::search(int)':
src/Node.h:28:34: warning: unused parameter 'pos' [-Wunused-parameter]
   28 |     virtual long long search(int pos) { return 0; }
      |                               ^~~~
[sudo] password for xxk:
[build] installed /usr/lib/mysql/plugin/libfhope.so
```

图 17: textbook 参数重新编译

在 textbook 参数下再次运行 same-number, 查询仍然得到 expected=20 actual=20 correct=YES, 唯一明文为 1、唯一密文和唯一编码均为 20, 但 leaf\_splits=0、internal\_splits=0、updates=0。这说明大容量、大编码空间下小规模数据难以触发结构变化, 但并不影响插入和查询正确性。

```
id=14 encoding=4467570830351532032 cipher_prefix=g9m9zfUBJkE8Q7+q2XNbRVsY plaintext=5
[query-check] expected=20 actual=20 correct=YES
[summary] unique_plaintexts=1 unique_ciphertexts=20 unique_inserted_encodings=20 leaf_splits=0 internal_splits=0 updates=0
[files] saved to results/textbook_same_number_20
```

图 18: textbook 参数下 same-number 查询正确但不触发分裂

## 6 实验问题与解决方法

### 6.1 UDF 状态需要重置

MySQL UDF 的编码树状态保存在数据库进程内。如果连续运行多个 workload 而不重置, 后续实验会继承前一轮树结构和分裂计数。为此, 本实验实现 FHReset(), 并在 pro\_reset() 中同时清空数据库表和重置编码树。每个 workload 启动时输出 local\_table cleared, MySQL table truncated, FH-OPE tree reset, 保证实验之间相互独立。

### 6.2 小规模实验不一定触发编码更新

编码更新只在局部编码间隙不足时发生。连续插入相同数字 5 能稳定验证频率隐藏和分裂, 但不必然触发重编码。本实验通过 increasing\_20 和 random\_small\_domain\_50 补充验证编码更新路径, 其中分别触发 4 次和 8 次编码更新, 从而覆盖了题目要求中的“观察编码更新”部分。

### 6.3 查询正确性不能只看返回顺序

数据库按 encoding 排序返回密文, 而随机插入和重编码可能改变结果顺序。因此, 本实验不使用列表顺序直接比较, 而是使用 Counter 比较期望明文集合与实际解密集集。只要多重集合一致, 即可判定范围查询语义正确。

## 7 实验结论

本实验完成了 FH-OPE 方案的核心复现，实现了 Python 客户端、MySQL UDF、C++ 编码树和 SQL 存储过程之间的完整协同。客户端通过随机 AES 加密避免相同明文产生重复密文，通过 `local_table` 和随机插入位置将相同明文分散到不同逻辑位置；服务端编码树维护可排序编码，并在节点容量达到阈值时执行叶节点或内部节点分裂，在编码空间不足时执行重编码和数据库同步更新。

主测试中，连续插入数字 5 共 20 次后，实验得到唯一明文数 1、唯一密文数 20、唯一编码数 20、叶节点分裂 7 次、内部节点分裂 2 次，查询 `[5,5]` 返回 20 条记录并通过正确性验证。这说明相同明文多次插入时，数据库侧无法通过重复密文或单一编码直接观察明文频率。

批量实验进一步验证了不同分布下的稳定性。所有 workload 的查询结果均正确；其中 `increasing_20` 和 `random_small_domain_50` 触发编码更新，证明重编码和 FHUpdate 同步逻辑有效。`textbook` 参数对照说明，大参数下小规模数据通常不触发分裂或编码更新，但仍保持查询正确。综合来看，本实验达到了“实现 FH-OPE、连续插入相同明文、观察分裂与编码更新、验证范围查询正确性”的实验目标。